Name: Ahmed Yasser Ali Refaat Ahmed El Sheikh

Neptun Code: ZLO1PI

Course name: programming 3

# Project Title: Hotel Booking Application

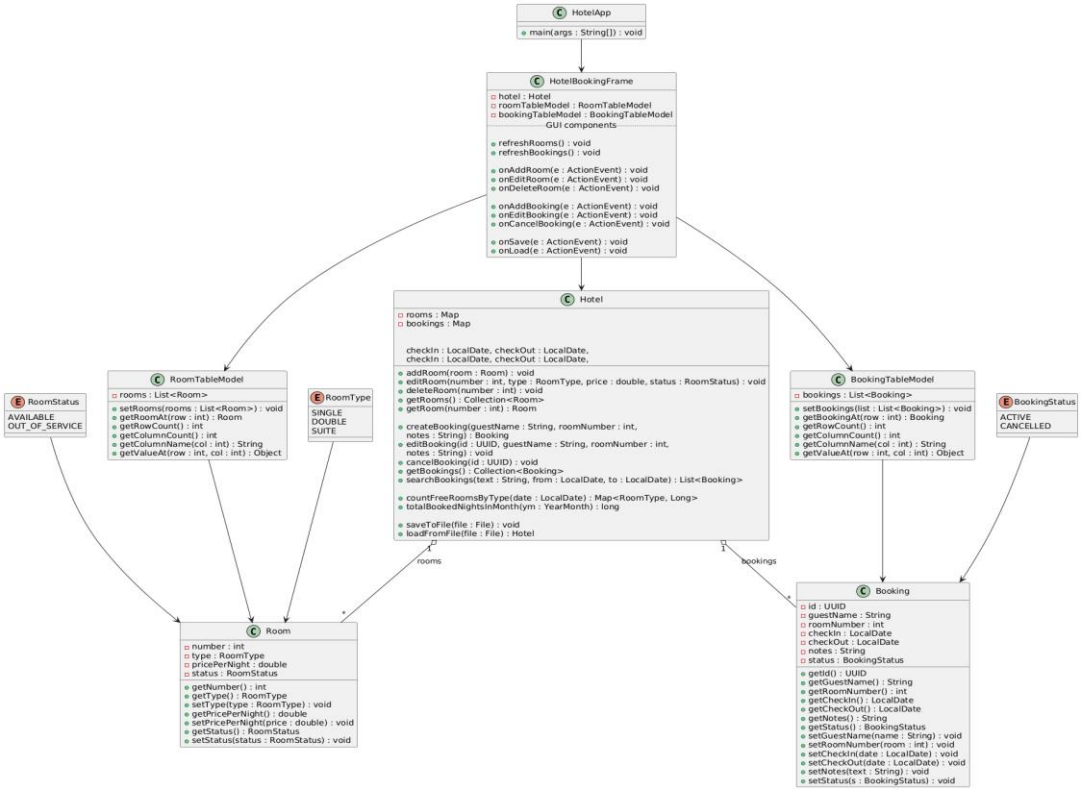
## Introduction:

The Hotel Booking System is a desktop-based Java application designed to simplify and automate the management of hotel reservations. Its primary goal is to provide an efficient way for hotel staff to handle room availability, guest bookings, reservation updates, and cancellations through a clear and user-friendly interface. By integrating a structured backend with an intuitive graphical interface built using Java Swing, the system ensures smooth operation for both daily use and long-term data handling.

At its core, the application models three fundamental components of hotel operations: **rooms**, **guests**, and **bookings**. Each room has a defined type, and each booking contains essential information such as guest name, stay dates, and reservation status. The system automatically checks room availability, prevents overlapping reservations, and maintains a reliable list of all bookings. To preserve data between sessions, the application uses Java serialization, allowing all room and booking information to be stored and reloaded seamlessly.

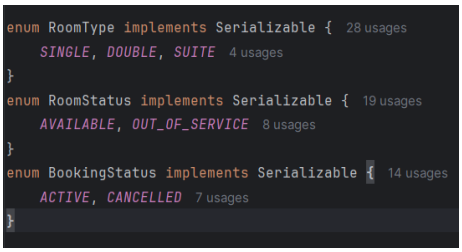
This project demonstrates clean object-oriented design through its use of classes, enums, and a dedicated booking manager responsible for the system's logic. The result is a practical, real-world application that shows key software engineering principles such as modularity, data validation, persistence, and UI-driven workflow.

**Class diagram**:



## Project composition:

In the project I used enums to eliminate string mistakes. I created roomtype enum to force only 3 room types (single, double, suite) while having another2 enums for room and booking status to see if they are available or not.




Room class:

The Room class models a physical hotel room. It contains a room number, a RoomType, a price per night, and a RoomStatus. These fields describe the room's identity, category, cost, and availability state. The class provides getters and setters for each field, allowing the hotel logic to modify room details as needed. This class is managed directly by the Hotel class and displayed in the GUI through the RoomTableModel.

Booking class:

The Booking class represents a guest reservation. It stores a unique booking ID (UUID), the guest's name, the room number they booked, check-in and check-out dates, any notes, and a BookingStatus indicating whether the booking is active or cancelled. The class provides getters and setters for all attributes. It is used by the Hotel class to check date conflicts, manage updates, and display booking information through the BookingTableModel.

Hotel class:

The Hotel class acts as the system's main logic and data manager. It stores all rooms and bookings using two maps: one mapping room numbers to Room objects, and the other mapping booking IDs to Booking objects. It provides full CRUD operations for rooms (add, edit, delete, retrieve) and for bookings (create, edit, cancel, list, search). It includes logic to detect overlapping bookings, ensure room availability, compute statistics such as free rooms by type or total booked nights in a month, and filter bookings by text or date range. Additionally, the Hotel class supports saving and loading all system data using Java serialization, allowing the application state to persist between runs.

## GUI Helper Classes

Room Table Model:

Room Table Model adapts the list of rooms into a form that is usable by a Swing JTable. It stores a list of Room objects and provides required table-model methods such as row and column counts, column names, and cell value access. It also includes a method to retrieve the Room object at a specific row. This class connects the GUI to the underlying hotel data structure.

Booking Table Model:

Booking Table Model performs the same function as Room Table Model, but for bookings. It stores a list of Booking objects and exposes them through standard table-model functions. It also provides a method to retrieve a booking by row index. The class ensures the JTable always displays accurate booking information whenever the data changes.

## GUI Classes

Hotel Booking Frame:

Hotel Booking Frame is the main graphical user interface for the application. It holds references to the Hotel object and both table models. It constructs the entire Swing-based interface, including tabs, buttons, tables, and dialogs. It handles all user-triggered actions, such as adding, editing, or deleting rooms and bookings, as well as loading and saving the hotel data. After each operation, it refreshes the tables to keep the displayed information synchronized with the underlying data.

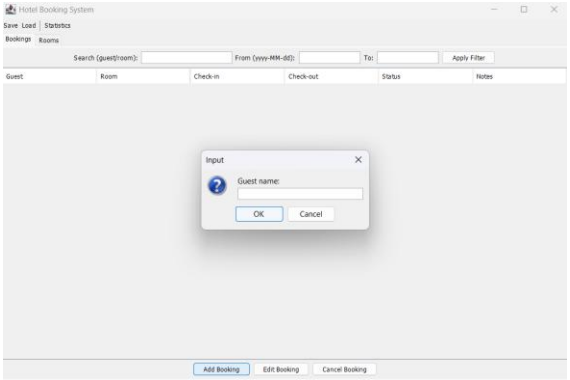
Hotel App:

The Hotel App class contains the program's main method. Its only job is to start the application by creating an instance of Hotel Booking Frame and making the GUI visible. It serves as the entry point of the software.

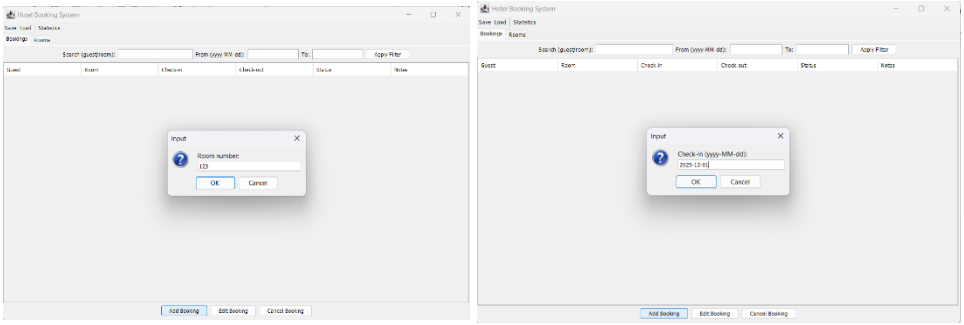
# Sequence diagrams

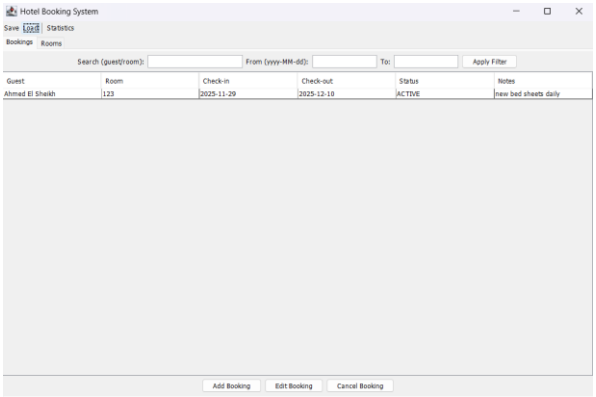
### Use Case: Add Booking

In the *Add Booking* use case, the interaction starts when the **User** presses the “Add Booking” button in the graphical interface. This click is handled by the HotelBookingFrame, which calls its own onAddBooking(...) event handler.



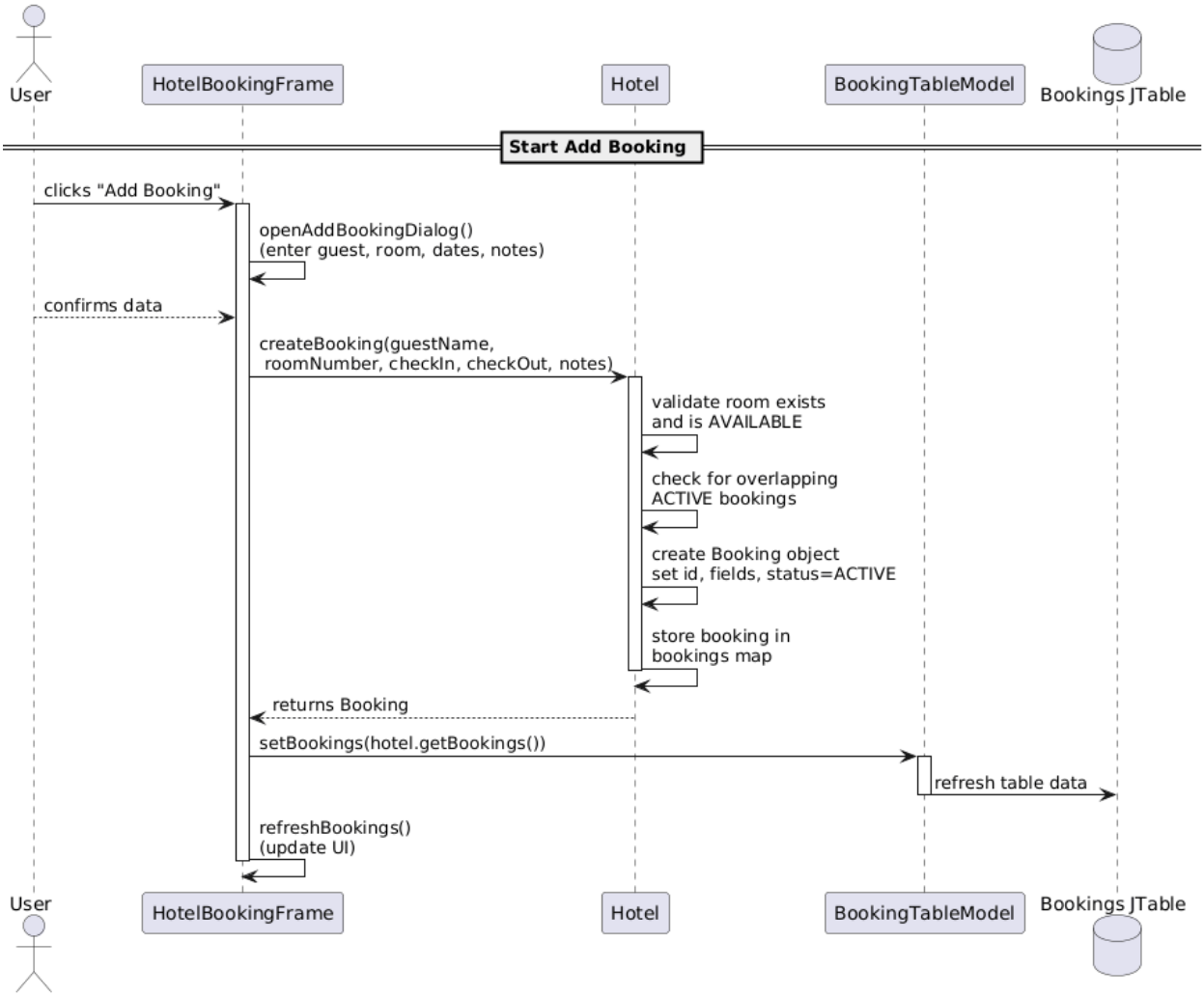

The frame then opens an input dialog or form where the user types the guest name, selects a room, and chooses check-in and check-out dates, optionally adding notes.



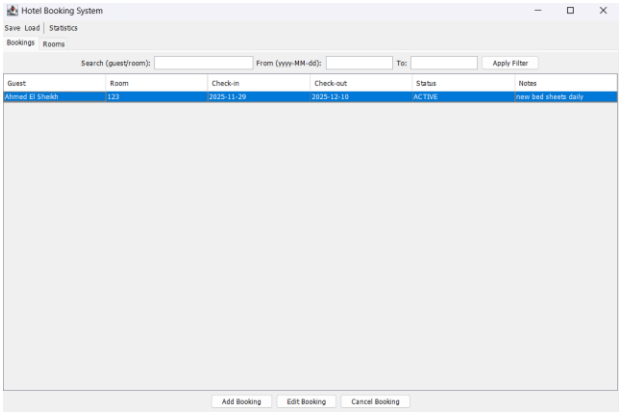

Once the user confirms the form, HotelBookingFrame validates the entered data at GUI level (for example, checking that fields are not empty) and then calls the createBooking(...) method on the Hotel object, passing the collected values. Inside Hotel, the method first

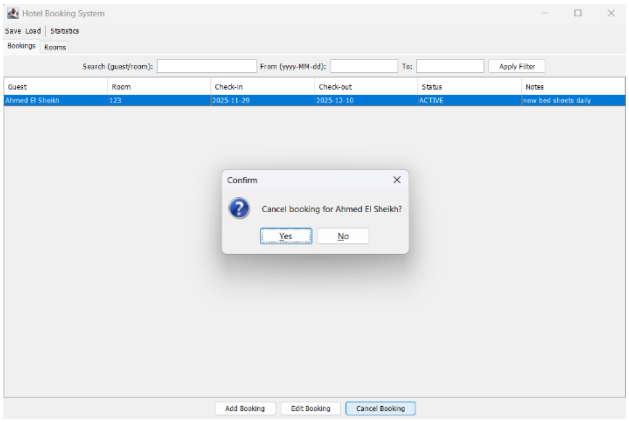
looks up the corresponding Room by room number and verifies that the room exists and is not out of service. Then Hotel checks for date conflicts by comparing the requested date range against existing ACTIVE bookings for the same room. If an overlap is found, Hotel throws an exception or returns an error that the HotelBookingFrame can show to the user. If there is no conflict, Hotel creates a new Booking object with a generated UUID, sets all of its fields (guest name, room number, dates, notes, initial status ACTIVE), and stores it in the bookings map. Control returns to HotelBookingFrame, which then updates the BookingTableModel by reloading the latest bookings from the Hotel and calling a refresh method such as refreshBookings(). Finally, the JTable on the screen is repainted with the new booking visible to the user, completing the use case.

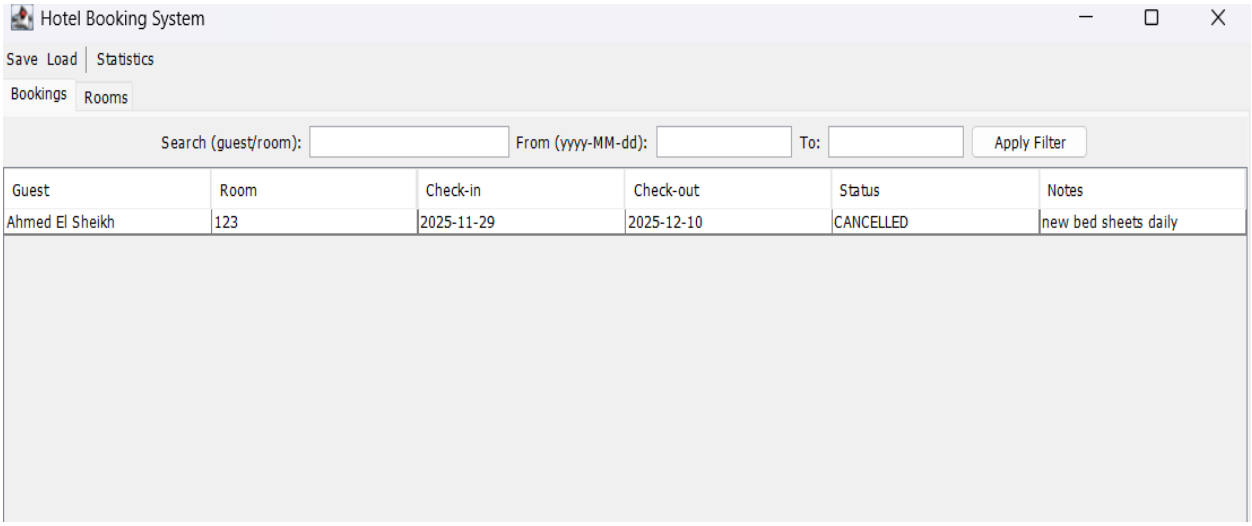


**Use case: cancel booking**

In the *Cancel Booking* use case, the sequence begins when the User selects an existing booking in the bookings table and clicks the “Cancel Booking” button.

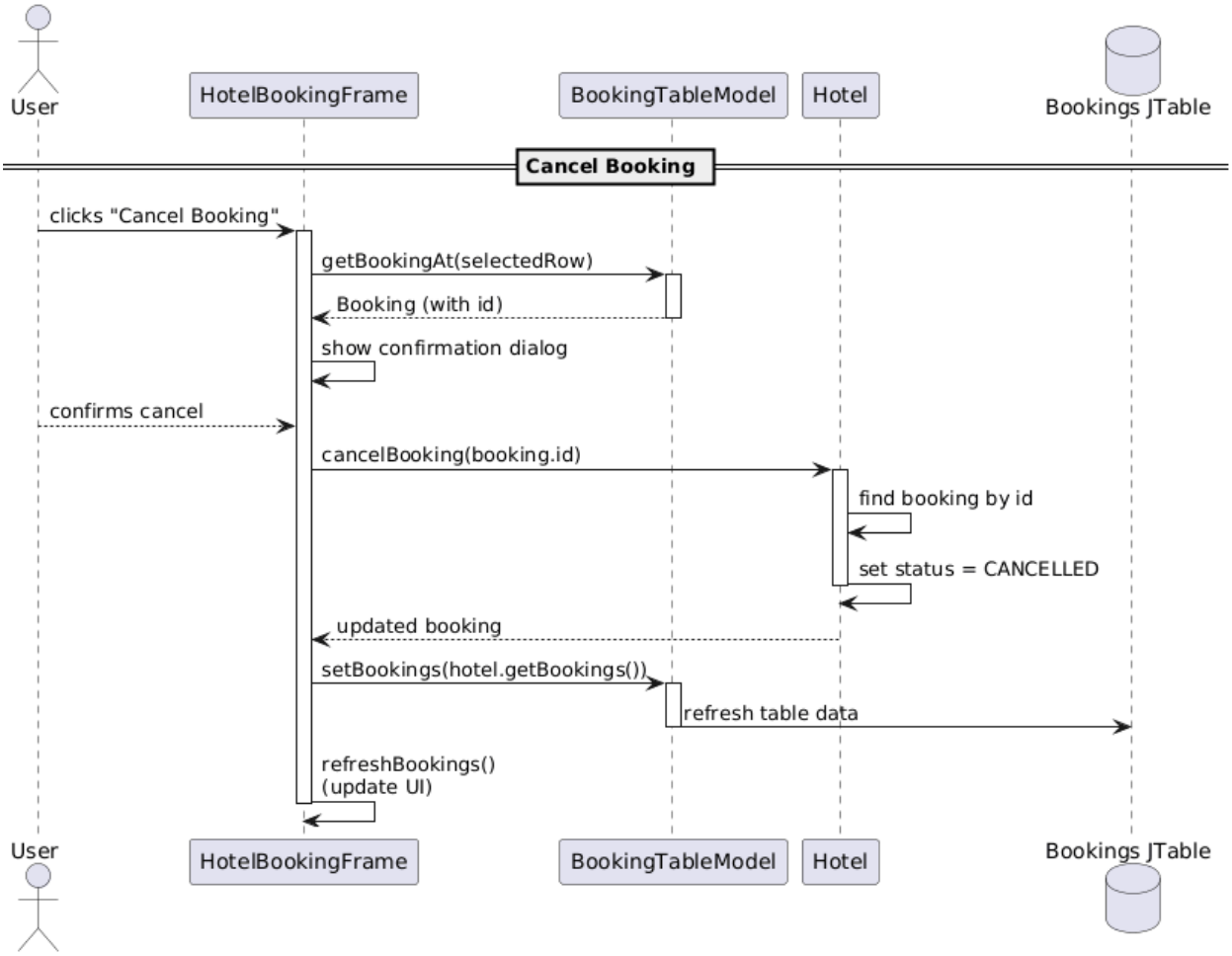





The HotelBookingFrame receives this event and its onCancelBooking(...) handler reads the selected row from the BookingTableModel. Using that row index, it retrieves the corresponding Booking object, typically via a method like getBookingAt(row). From this booking, the frame obtains the booking’s unique ID (UUID) and may optionally ask the user for confirmation (for example with a confirmation dialog). If the user confirms, HotelBookingFrame calls the cancelBooking(id) method on the Hotel object. Inside Hotel, this method looks up the booking in its internal bookings map by ID. If found, it changes the booking’s status field from ACTIVE to CANCELLED, leaving the rest of the booking data (guest, dates, room) unchanged so that history is preserved. After the status change, control returns to HotelBookingFrame, which then refreshes the data in the BookingTableModel, again typically through a refreshBookings() method that reloads the full list of bookings from Hotel. The associated JTable is updated to reflect that this booking

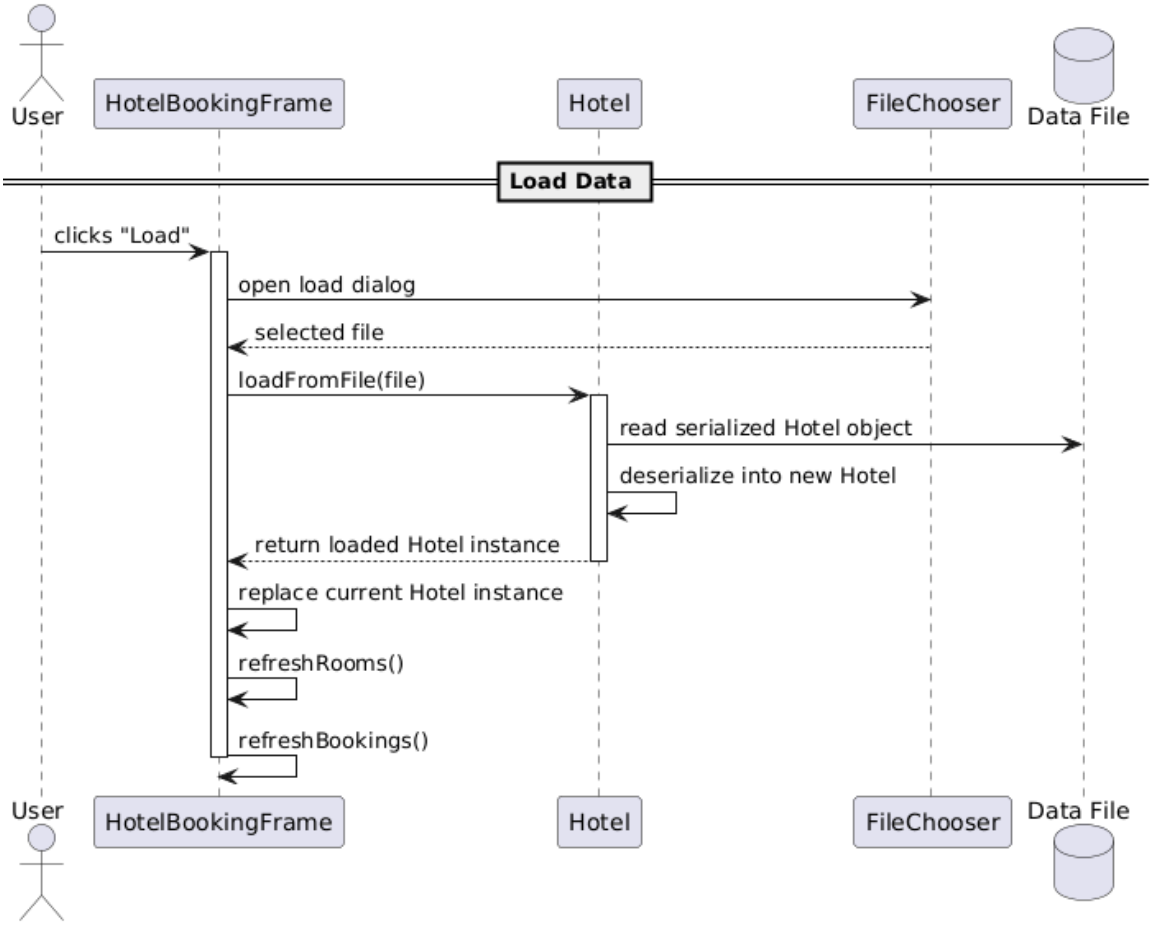
is now cancelled (for example, by showing the status column as CANCELLED), and the use case ends with the user seeing the updated status on the screen.



## Saving and loading data:

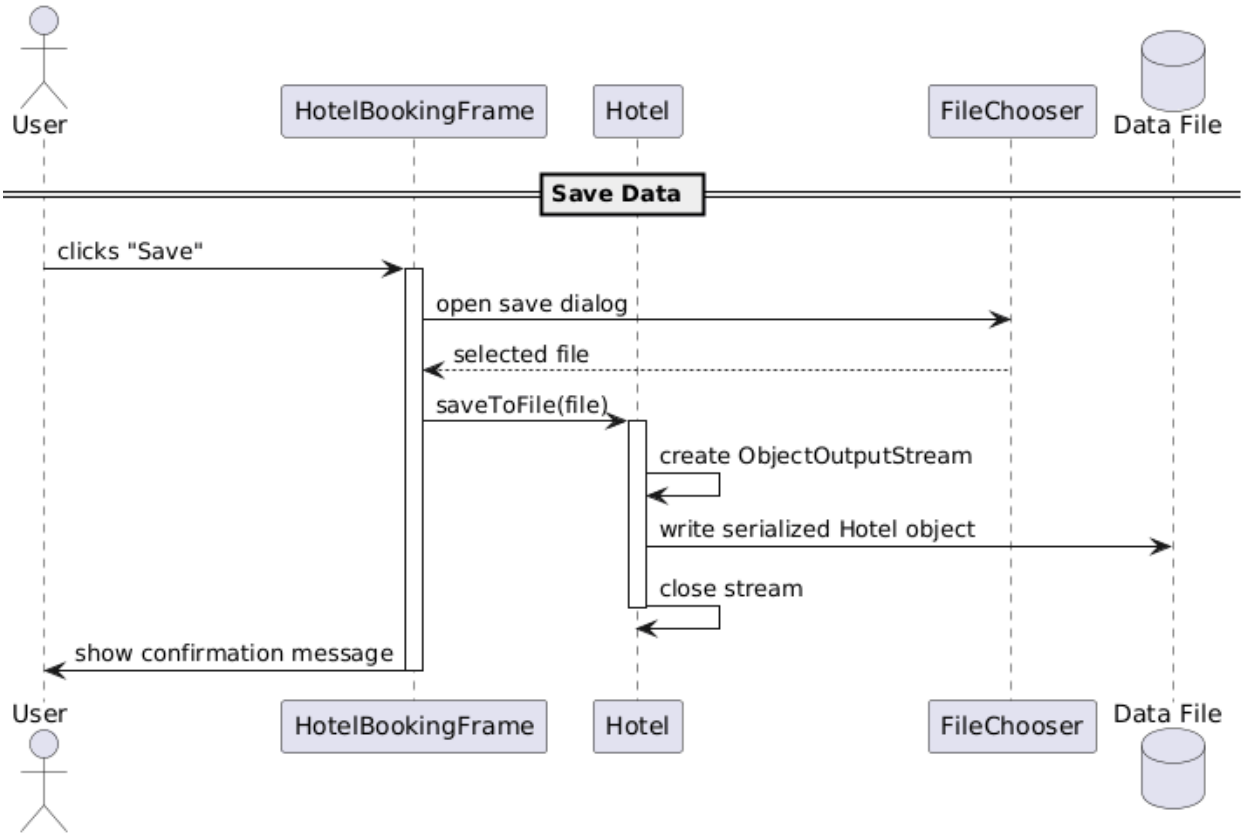
### Use Case: Save Data

In the *Save Data* use case, the interaction begins when the **User** selects the “Save” option in the graphical interface. The HotelBookingFrame receives this action and triggers its onSave(...) method. The frame typically opens a file chooser dialog to allow the user to select a destination file. Once the user confirms the location, HotelBookingFrame calls the saveToFile(file) method on the Hotel object. Inside the Hotel class, this method creates an ObjectOutputStream and serializes the entire Hotel instance, including all rooms and bookings, into the selected file. This process writes the current application state—such as room details, booking history, and statuses—into persistent storage so it can be restored later. After saving completes, control returns to the GUI, where a confirmation dialog informs the user that the data has been successfully saved.



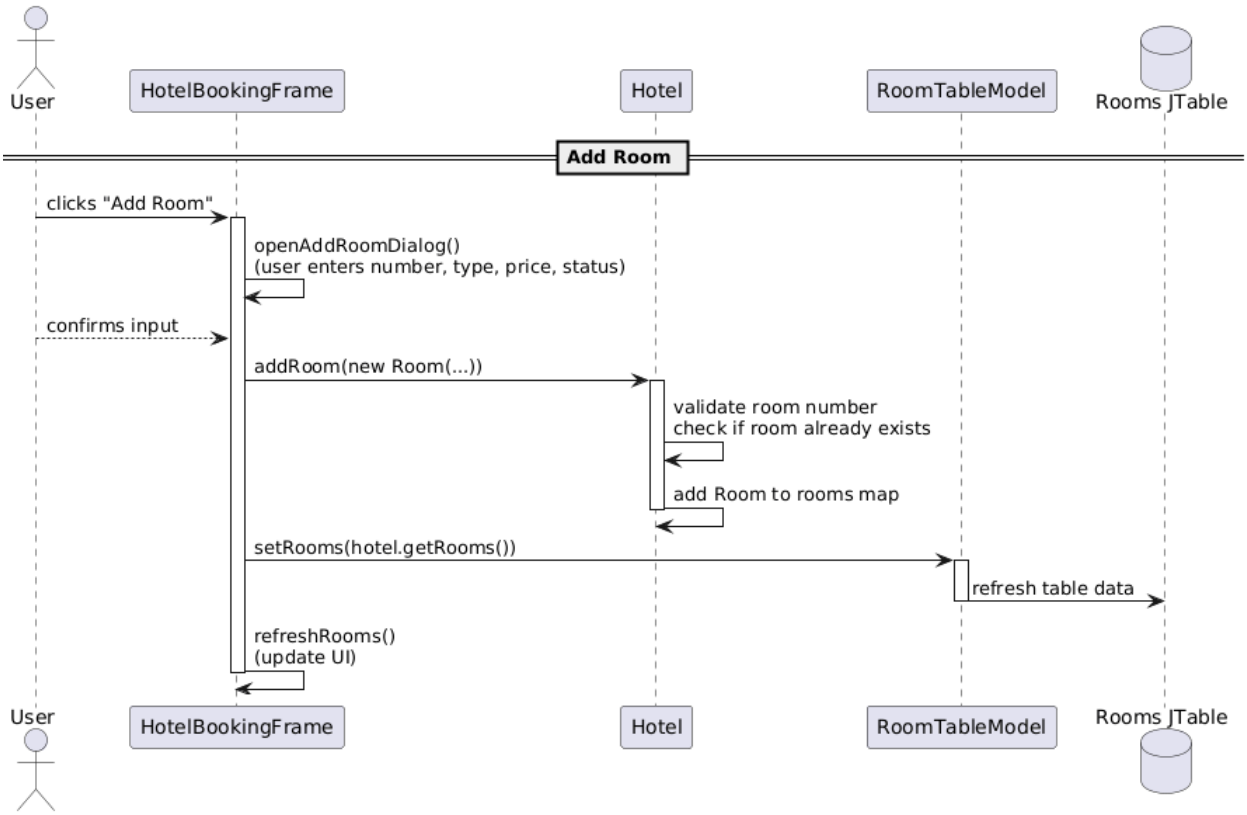
**Use Case: Load Data**

In the *Load Data* use case, the **User** selects the “Load” option from the menu or toolbar in the GUI. The HotelBookingFrame responds by opening a file chooser dialog for the user to pick a previously saved data file. When the user confirms, the frame calls the static method Hotel.loadFromFile(file), which opens an ObjectInputStream and deserializes a Hotel object from disk. This creates a complete in-memory reconstruction of the hotel’s data, including all rooms, bookings, and their statuses. The newly loaded Hotel instance is then stored inside the HotelBookingFrame, replacing the old one. After loading, the frame calls refreshRooms() and refreshBookings() to update both tables so the GUI accurately reflects the restored data. The use case concludes with the updated hotel data now visible and ready for further interaction.



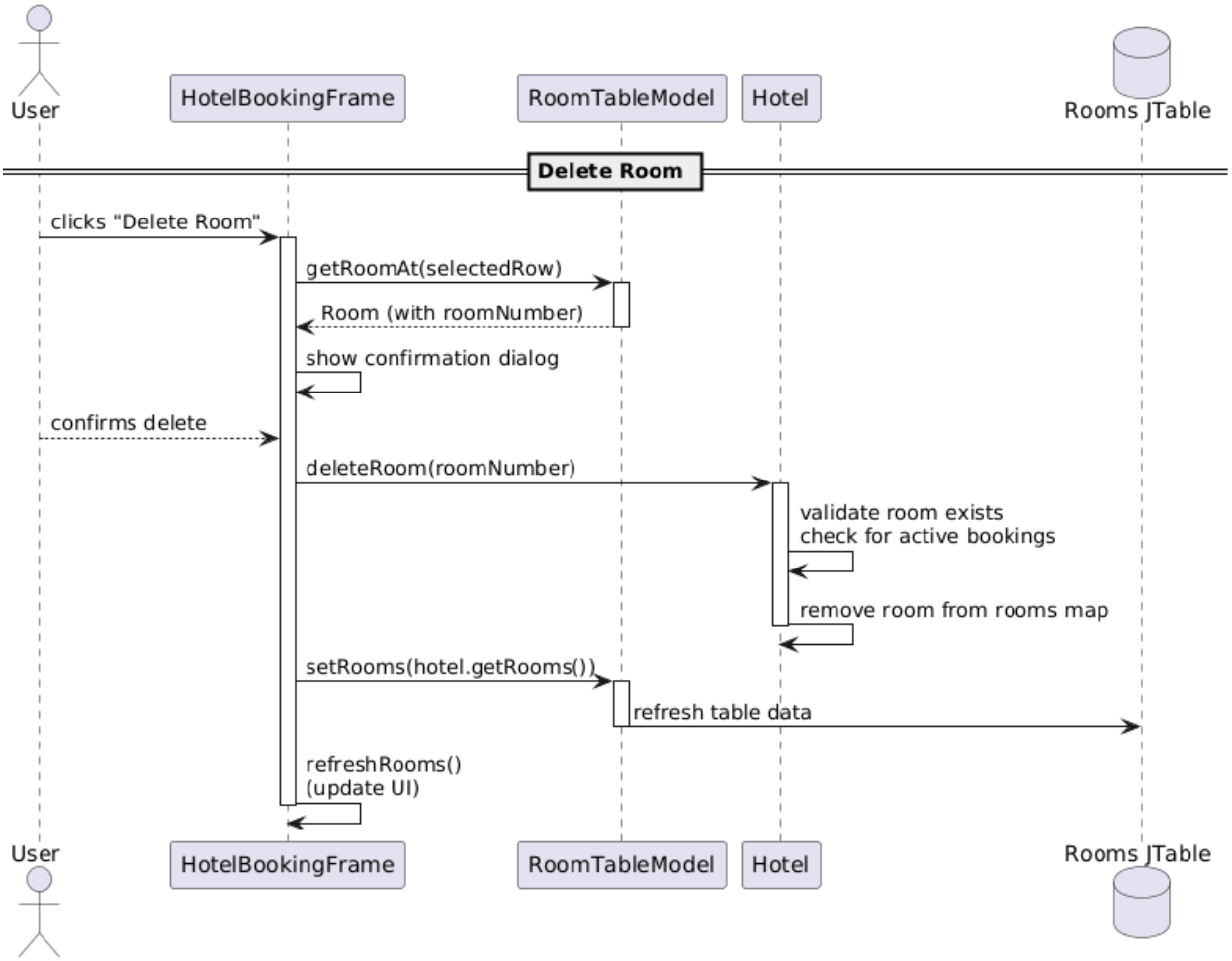
**Use case: add room**

In the *Add Room* use case, the process begins when the User selects the “Add Room” option in the graphical interface. The HotelBookingFrame handles this action through its onAddRoom(...) method and opens a dialog where the user can enter the room number, select a room type, choose a room status, and specify the price per night. After the user confirms the input, the frame performs simple validation such as ensuring that mandatory fields are filled and that the room number is numeric. It then calls addRoom(new Room(...)) on the Hotel object, passing a newly constructed Room instance. The Hotel class checks whether a room with the same number already exists and, if not, adds the new room to its internal rooms map. After successful insertion, control returns to HotelBookingFrame, which updates the RoomTableModel by reloading the current list of rooms and refreshing the associated JTable. The use case concludes with the newly added room immediately visible in the interface.



**Use case: delete room**

In the *Delete Room* use case, the interaction begins when the **User** selects an existing room in the rooms table and clicks the “Delete Room” button. The HotelBookingFrame receives this event in the onDeleteRoom(...) method. The frame retrieves the selected room from the RoomTableModel using a method such as getRoomAt(row) and typically asks the user for confirmation before proceeding. Once confirmed, the frame calls the deleteRoom(roomNumber) method on the Hotel object. Inside the Hotel class, this method validates that the room exists and may also check whether the room is associated with any active bookings to prevent accidental removal. If deletion is allowed, the room is removed from the rooms map. Control then returns to the GUI, where HotelBookingFrame updates the RoomTableModel with the new room list and refreshes the table so the deleted room is no longer displayed. The use case ends with the updated list of rooms shown to the user.



## Description of unit testing:

The JUnit tests included in this project verify the correctness of the core business logic contained in the Hotel class, which manages rooms, bookings, and date-based operations. The test suite begins by creating sample hotel instances with predefined rooms to ensure predictable results. It then checks successful booking creation by confirming that newly created bookings contain the correct guest information, room number, and dates, and that the booking is stored in the hotel's internal data structure. Another test verifies that the system correctly rejects overlapping bookings by attempting to create two reservations for the same room and dates, expecting an exception to be thrown. Additional tests validate booking cancellation, ensuring that the booking's status is changed to CANCELLED without removing the entry. The search functionality is tested by adding multiple bookings and confirming that filtering by guest name or date range returns the correct matching results. The suite also includes tests for statistical functions, such as counting free rooms by type on a specific date and calculating the total booked nights within a month, verifying that these methods return accurate numeric results. Together, these JUnit tests ensure that critical operations—booking management, conflict detection, searching, cancellation, and analytics—work reliably and consistently, reinforcing the correctness and stability of the application's backend logic.

**1. testCreateBooking_Success**

This test verifies that the booking creation process works correctly under normal conditions. It creates a sample hotel with predefined rooms, then creates a booking with valid guest information and a valid date range. The test asserts that the returned booking object is not null, that it has a generated UUID, and that all fields—guest name, room number, check-in date, and check-out date—match the input values. It also checks that the booking is correctly added to the hotel's internal list of bookings. This ensures the core booking creation workflow behaves as expected.

---

**2. testCreateBooking_OverlappingDatesFail**

This test checks that the system properly rejects conflicting bookings. It first adds a valid booking to a room, then attempts to add a second booking for the same room with overlapping dates. The test expects an IllegalStateException to be thrown. This verifies the hotel's date-conflict detection logic, ensuring that the system prevents double-booking a room for the same time period.

---

**3. testCancelBooking_ChangesStatusToCancelled**

This test confirms that canceling a booking updates only the booking's status without removing it. After creating a valid booking, the test calls the cancelBooking(id) method and then checks that the booking's status has been changed to CANCELLED. This ensures that the system preserves booking history while marking reservations as inactive.

---

**4. testSearchBookings_ByTextAndDate**

This test validates the booking search functionality. It creates two bookings with different names and date ranges, then performs two types of searches:

- A text-based search that looks for a guest name (case-insensitive).
- A date-range search that filters bookings within a specific month.
  The test ensures that each search returns only the correct matching bookings, verifying that the search algorithm works accurately for both text and date filtering.

---

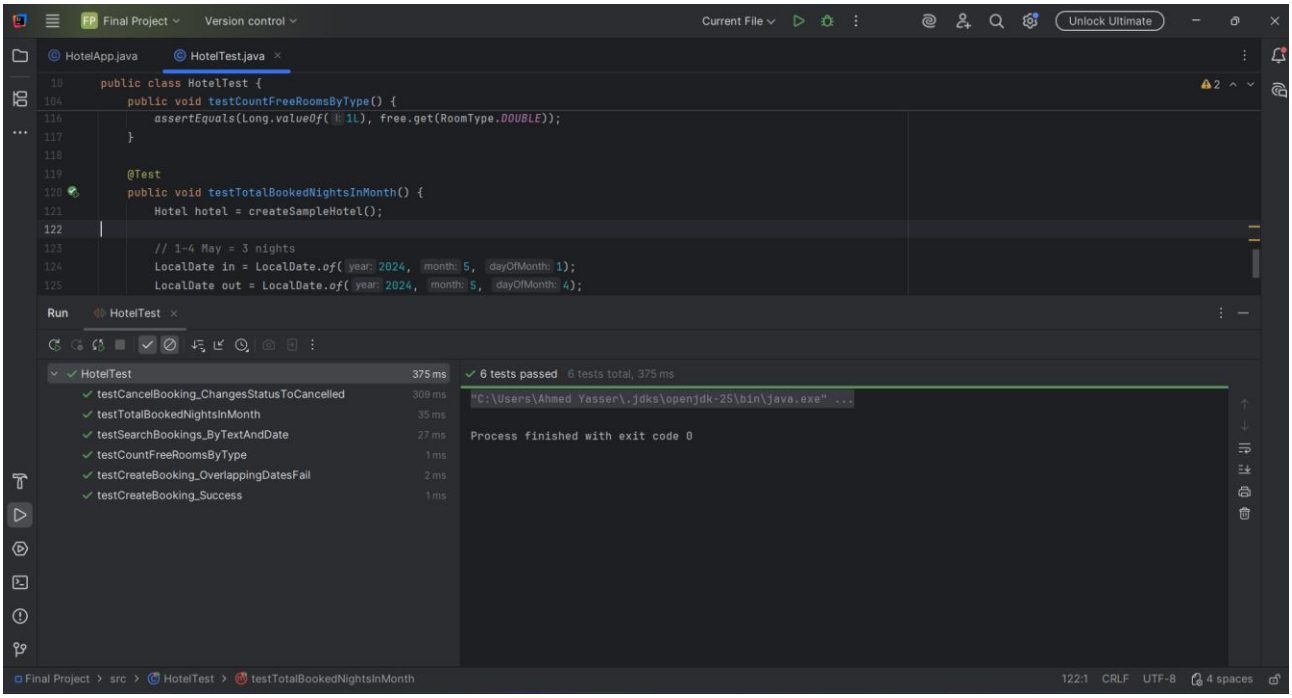
**5. testCountFreeRoomsByType**

This test checks the hotel's ability to calculate how many rooms of each type are available on a given date. After creating rooms and booking one of them for a specific date, the test calls countFreeRoomsByType and verifies that the occupied room's type returns a count of zero while other room types return the correct available count. This confirms that the occupancy calculation logic is accurate.

---

**6. testTotalBookedNightsInMonth**

This test ensures that the system correctly calculates the total number of booked nights across all bookings within a specific month. After creating a booking that spans several days, the test calls totalBookedNightsInMonth(YearMonth) and asserts that the returned value matches the number of nights booked. This validates the date-based analytics functionality of the hotel system


Final Project
Version control
Current File
Unlock Ultimate
HotelApp.java
HotelTest.java
public class HotelTest {
public void testCountFreeRoomsByType() {
assertEquals(Long.valueOf( l: 1L), free.get(RoomType.DOUBLE));
}
@Test
public void testTotalBookedNightsInMonth() {
Hotel hotel = createSampleHotel();
// 1-4 May = 3 nights
LocalDate in = LocalDate.of( year: 2024, month: 5, dayOfMonth: 1);
LocalDate out = LocalDate.of( year: 2024, month: 5, dayOfMonth: 4);
Run
HotelTest
HotelTest 375 ms
testCancelBooking_ChangesStatusToCancelled 309 ms
testTotalBookedNightsInMonth 35 ms
testSearchBookings_ByTextAndDate 27 ms
testCountFreeRoomsByType 1 ms
testCreateBooking_OverlappingDatesFail 2 ms
testCreateBooking_Success 1 ms
6 tests passed 6 tests total, 375 ms
"C:\Users\Ahmed Yasser\.jdks\openjdk-25\bin\java.exe" ...
Process finished with exit code 0
Final Project › src › HotelTest › testTotalBookedNightsInMonth
122:1 CRLF UTF-8 4 spaces